Nostrian Conquest

Sysop Manual

A Rust recreation inspired by the classic 1990s BBS door game Esterian Conquest.

Version 1.0.0-beta.3 — Beta

Revision date: October 14, 2026



This manual © 2026 Mason A. Green and is licensed under CC BY-NC-SA 4.0.

License text: https://creativecommons.org/licenses/by-nc-sa/4.0/

Contents

1. Introduction 2
2. Getting Started: Hosting a Campaign 2
   - 2.1. System Requirements 2
   - 2.2. Building from Source 2
   - 2.3. Package Distribution 2
   - 2.4. Choose Your Deployment 3
     - 2.4.1. Localhost Session 3
     - 2.4.2. BBS Door Host 3
   - 2.5. Game Directory Layout 3
   - 2.6. Initializing a New Game 4
3. Game Directory Structure 4
4. Configuration 4
   - 4.1. Direct `nc-game` 5
   - 4.2. BBS Door Host 5
   - 4.3. BBS `config.kdl` 5
   - 4.4. Display Defaults 6
5. Terminal Access 6
6. Localhost Session Setup 6
7. BBS Door Setup 6
   - 7.1. Flags for Door Mode 7
   - 7.2. Drop File 7
   - 7.3. Verified BBS Hosts 8
8. File-Based Turn Submission 9
9. Turn Processing and Maintenance 10
10. Player Management 10
    - 10.1. Reserving Seats 11
11. Terminology 11
12. CLI Reference 12
    - 12.1. nc-sysop 12
    - 12.2. nc-game and nc-door 12

1. Introduction

Nostrian Conquest is a multi-player strategy game inspired by a DOS-era BBS classic. This Rust edition gives the sysop a direct player client, a BBS door, and the tools needed to run a campaign.

This manual is for the **sysop** — the man who hosts the game. It covers:

- choosing the live deployment path
- creating and maintaining a game directory
- running a local game
- putting `nc-door` on a BBS
- managing players and yearly maintenance

For player-facing rules and gameplay, see the **Player Manual**.

To reach the author with questions, bug reports or ideas, message this Nostr public key. The contact page also lists Discord and Matrix.

`npub1cecl2g9z776jvpudpqxajsrsgwuvq4vp7w26q74tx2438zy3q9vq8st7pp`

This manual is the authoritative sysop manual for the Rust edition. The preserved `.DOC` files in `original/v1.5/` remain historical reference material and an ambiguity fallback for classic operator intent.

2. Getting Started: Hosting a Campaign

2.1. System Requirements

- Linux, macOS, or Windows
- Rust toolchain (stable) for normal localhost builds
- A terminal emulator for localhost play
- A BBS server with socket support only if you specifically want legacy door deployment

2.2. Building from Source

From the repository root:

```
cd rust
cargo build --release -p nc-sysop -p nc-game
```

The release binaries will be in `rust/target/release/`.

> **IMPORTANT:** If you update to a newer build and the game reports that the runtime schema is unsupported, the `ncgame.db` in that directory is stale for that build. Recreate or refresh `ncgame.db` before launching again. `nc-door` now shows this as an in-app error screen instead of dropping straight back to the shell.

2.3. Package Distribution

BBS sysops may use a packaged `nc-sysop` archive or build from source. Localhost sysops build from source and run `nc-game` directly.

Keep the binary roles straight.

`nc-game` is the direct localhost session client. `nc-door` is the BBS door entrypoint on Windows and Linux. `nc-sysop` is the administrator's tool for creation, settings, reservations, and maintenance.

The public Nostrian packages do not bundle preserved Esterian Conquest executables, manuals, or DOS helper assets.

2.4. Choose Your Deployment

Use one of these live paths today.

2.4.1. Localhost Session

Use this for direct same-machine play, hotseat testing, or a small trusted session.

1. Build the binaries from source.
2. Create a game directory with `nc-sysop new-game`.
3. Run `nc-game` directly.
4. Run `nc-sysop maint` when the year should advance.

The simplest localhost setup is:

```
nc-sysop new-game /home/sysop/nc-games/friday-night --name "Friday Night NC" --players 4
nc-game --dir /home/sysop/nc-games/friday-night --player 1
```

2.4.2. BBS Door Host

Use this when you want `nc-door` under Mystic, ENiGMA½, Synchronet, WWIV, or a similar BBS.

1. Create the game directory.
2. Write a minimal per-game `config.kdl` with `players` and any fixed-seat `reservations`.
3. Initialize it with `nc-sysop new-game --bbs`.
4. Launch `nc-door` with a BBS dropfile.
5. Keep maintenance outside the door. Run `nc-sysop maint` from the host or the BBS event runner.

For BBS hosting, use the packaged `nc-sysop` archive or build from source.

> **IMPORTANT:** A planned Nostr/GameServer path exists, but it is not part of the live `nc-sysop` workflow today. For live games, use localhost or BBS hosting.

For the exact launcher setups, see:

- `docs/sysop/bbs/mystic-bbs-setup.md`
- `docs/sysop/bbs/enigma-bbs-setup.md`
- `docs/sysop/bbs/synchronet-bbs-setup.md`
- `docs/sysop/bbs/wwiv-bbs-setup.md`

2.5. Game Directory Layout

Direct `nc-game` campaigns are DB-only:

```
/path/to/mygame/
  ncgame.db
```

BBS door campaigns keep a minimal live config file beside the runtime DB:

```
/path/to/mygame/
  config.kdl
  ncgame.db
```

All tools take `--dir /path/to/mygame` to locate the game.

2.6. Initializing a New Game

Create a new game with `nc-sysop new-game`:

```
sudo -u ncgame nc-sysop new-game /srv/nc/games/friday-night --name "Friday Night NC" --players 4
```

This creates one runtime file: `ncgame.db`.

For BBS door campaigns, write `config.kdl` first and then initialize with:

```
nc-sysop new-game --bbs /path/to/mygame
```

Use these creation flags:

- **`--name`:** String. Optional game title stored in `ncgame.db`. If omitted, `nc-sysop` derives a title from the directory slug.
- **`--players`:** Integer. Number of empires. Supported range: `1-25`. Default: `4`.
- **`--seed`:** Integer. Optional campaign RNG seed. It controls map layout, starting positions, and all random events. If omitted, the engine picks a random seed and saves it to `ncgame.db`. The seed cannot be changed after creation.
- **`--bbs`:** Flag. Initialize a BBS campaign from an existing per-game `config.kdl`. In this mode, `--name` and `--players` are not accepted on the command line. Use `--seed` only if you intentionally want a reproducible map for that one campaign.

> **NOTE:** Use a different seed for every game. Reusing the same seed produces the same map, starting positions, and event sequence every time. For BBS campaigns, keep that override on the `new-game --bbs --seed ...` command line instead of storing it in `config.kdl`.

The target directory basename becomes the stable game slug. It must use only lowercase ASCII letters, digits, and dashes. The slug is distinct from the human-readable `game_name`.

3. Game Directory Structure

- `ncgame.db`: The SQLite runtime database. All live game state is here. Do not edit it by hand.
- `config.kdl`: Present only for BBS door campaigns. It holds `players` and optional seat `reservations`.

4. Configuration

Keep the live paths separate:

4.1. Direct `nc-game`

- Live state: per-game `ncgame.db`
- Runtime path: `nc-game --dir ... --player ...` on localhost
- Operator rule: use the normal `nc-sysop settings ...` surface and schedule `nc-sysop maint` yourself

4.2. BBS Door Host

- Live state: per-game `config.kdl` plus `ncgame.db`
- Runtime path: `nc-door` with a dropfile; ENiGMA½ also passes `--socket-port` and Windows Synchronet passes `--socket-descriptor`
- Operator rule: BBS `config.kdl` supports only `players` and `reservations`; `settings set` does not apply
- Common hosts: Mystic, ENiGMA½, and Synchronet on Windows or Linux

Use `nc-sysop settings ...` to inspect any non-BBS campaign:

```
nc-sysop settings show --dir /srv/nc/games/friday-night
nc-sysop settings set --dir /srv/nc/games/friday-night --game-name "Friday Night NC"
nc-sysop settings reserve --dir /srv/nc/games/friday-night --player 1 --alias SYSOP
```

For the full non-BBS settings reference, including the raw `settings show` shape and the advanced carried-forward state fields, see `docs/sysop/rust/campaign-settings.md`.

4.3. BBS `config.kdl`

BBS campaigns use a minimal live per-game `config.kdl` instead of the non-BBS settings surface:

```
players 4
reservations {
  seat player=1 alias="SYSOP"
  seat player=2 alias="NightShade"
}
```

Supported BBS fields are only:

- `players`
- `reservations`

`nc-sysop settings show --dir /path/to/mygame` reports BBS campaigns in a different shape:

```
mode=bbs
players=4
reservation seat=1 alias=SYSOP
reservation seat=2 alias=NightShade
```

Do not put `game_name`, `theme`, `snoop`, `session`, `inactivity`, or `maintenance` in the BBS file. Do not put `seed` there either. It is a one-shot `new-game` command line override, not live BBS config.

For BBS campaigns, `nc-sysop settings reserve` and `settings unreserve` edit this file for you. `settings set` is for non-BBS campaigns.

4.4. Display Defaults

The default display look is controlled by `default_theme_key`. That key names a compiled-in color set. It is not a file path.

Players may still choose a local color theme inside `nc-game`. That choice is stored in `ncgame.db` as a player preference.

In BBS door mode, `nc-door` does not use `default_theme_key` at runtime. Door sessions always force the bundled `mag16` palette so ANSI16 terminals and BBS clients get a predictable baseline.

5. Terminal Access

`nc-game` is a direct terminal client. The primary supported path is local same-machine play, but if you manually remote into a trusted shell and launch `nc-game` there, it behaves the same way. No special session flags are required.

Color mode is auto-detected from the environment:

- `COLORTERM=truecolor` → 24-bit RGB
- `TERM` containing `256color` → 256-color
- Otherwise → 16-color ANSI fallback

Force a specific mode with `--color-mode` if your terminal setup does not propagate `COLORTERM` reliably:

```
nc-game --dir /path/to/mygame --player 1 --color-mode 256
```

UTF-8 encoding (the default) is correct for normal terminals. Use `--encoding cp437` only when you are proxying through a BBS or a CP437 terminal emulator over SSH.

6. Localhost Session Setup

Use these steps for localhost play:

1. Create or choose a game directory.
2. Run `nc-game` directly in your terminal:

```
nc-game --dir /path/to/mygame --player 1
```

3. Run `nc-sysop maint /path/to/mygame` when you want the year to advance.

Color mode and encoding default to `auto` / `utf8`. The client reads `COLORTERM` and `TERM` automatically.

7. BBS Door Setup

`nc-door` is the standard BBS entrypoint on Windows and Linux. Use it when the host is a BBS. Keep `nc-game` for direct localhost sessions. In door mode, `nc-door` uses CP437 and 16-color ANSI for classic terminal clients.

7.1. Flags for Door Mode

```
nc-door --dir /path/to/mygame --dropfile /path/to/DOOR32.SYS --encoding cp437 --color-mode ansi16
```

`--encoding cp437` switches box-drawing characters and other extended characters to the CP437 code page, which is required for correct rendering in classic ANSI-aware BBS terminals (SyncTERM, NetRunner, etc.).

When `--encoding cp437` is active, `--color-mode` defaults to `ansi16` automatically. Override only if you know your BBS clients support a richer color depth.

Door sessions also force the bundled `mag16` theme regardless of the campaign's normal `default_theme_key`. The in-game **A>nsi color ON/OFF** command toggles between that ANSI16 palette and a greyscale monochrome projection.

7.2. Drop File

`nc-door` can read a BBS drop file directly with `--dropfile <path>`:

```
nc-door --dir /path/to/mygame --dropfile /path/to/DOOR32.SYS
```

Supported drop file formats (auto-detected by filename, case-insensitive):

- `DOOR32.SYS` — modern standard (Enigma, Mystic, Talisman, etc.)
- `DOOR.SYS` — legacy, widest BBS software support
- `CHAIN.TXT` — WWIV format

The drop file supplies the player alias and session time limit. Explicit CLI flags always override drop file values. Drop files may be written in CP437 or UTF-8. When `--dropfile` is given and `--encoding` is not, encoding defaults to `cp437` automatically.

`DOOR.SYS` supplies the handle from line 36, or the real name from line 10 when line 36 is missing or blank. When the handle matches no seat, the real name is tried. It resumes a seat reserved under the real name, or a joined seat not yet bound by handle; that seat then takes the caller's handle. A seat is bound by handle once its player joins or enters it with a handle, and from then on the real name never matches it, so a caller whose real name equals another player's handle cannot enter that player's empire.

A `DOOR.SYS` with no handle on line 36 gives only the real name. When that name matches the handle of a seat bound by handle, `nc-door` refuses the login. The caller sees that the login could not be matched and that the sysop has been notified. `nc-door` writes the real name and the seat number to standard error, and to the log when `--log-file` is set, then exits with status 1. To fix it, set the BBS to write handles on line 36. If the caller owns that seat, you can instead reserve it under the real name:

```
nc-sysop settings reserve --dir /path/to/mygame --player 2 --alias "Real Name"
```

`--timeout <minutes>` sets a session time limit independently of a drop file.

In door mode the time limit is enforced: when it runs out, `nc-door` shows a short message and exits normally, just as if the caller had quit. Game state is saved as each action is applied, so nothing is lost. When the limit is longer than two minutes, the caller sees a one-

line warning two minutes before it runs out, on the row above the command line; the next screen redraw clears it. The time limit counts from door launch. A non-BBS campaign played through door mode also ends the session after `session_max_idle_minutes` without a keypress. BBS campaigns have no idle setting; the time limit bounds an abandoned session instead.

A session that stays open across a maintenance run cannot save into the finished year. Its next save is refused, and the caller is told "Maintenance has run; your last change was not saved. Log in again." In door mode `nc-door` shows that line and exits normally, as it does at the time limit. A local `nc-game` or `nc-helm` session shows the line and stays open without saving. `nc-game submit-turn` fails with the same message if maintenance runs between its load and its save.

Reserve seats in the BBS campaign's `config.kdl` when you want the caller alias to determine the empire automatically. You can either edit the file directly or use:

```
nc-sysop settings reserve --dir /path/to/mygame --player 1 --alias SYSOP
nc-sysop settings reserve --dir /path/to/mygame --player 2 --alias NightShade
```

With that in place, a reserved caller can launch with `--dropfile` alone. If the caller alias is not reserved, `nc-door` still uses the same dropfile-only door flow:

- if the alias already matches a stored joined player handle, that caller resumes the matching empire automatically
- if the alias does not match an existing joined empire, the caller lands on the BBS first-time menu and can claim the lowest-numbered open unreserved empire only when the join is actually confirmed
- if the game is full, the caller still reaches the BBS first-time menu, but `J` is refused with the normal no-open-empires notice

> **NOTE:** The original DOS `ECGAME.EXE` v1.5 expects a strict 32-line WWIV-style `CHAIN.TXT` drop file. Enigma BBS-generated `DOOR.SYS` / `DORINFO` files will crash `ECGAME.EXE`. See the legacy DOS compatibility section in `docs/sysop/bbs/enigma-bbs-setup.md` if you need to host the original binary.

7.3. Verified BBS Hosts

The native Rust `nc-door` binary is now verified on Mystic, ENiGMA½, and Synchronet, and WWIV now has a validated Linux path. The shared core launch shape is simple: pass `--dir` and a dropfile. ENiGMA½ also passes `--socket-port`, and Windows Synchronet also passes `--socket-descriptor`. The helper wrapper at `tools/bbs/run_nc_rust.sh` remains useful only for source-tree testing and is not the normal packaged door path.

Mystic uses the direct dropfile path:

```
nc-door --dir /path/to/mygame --dropfile /path/to/DOOR32.SYS
```

ENiGMA½ uses the same dropfile plus its temporary localhost socket server:

```
nc-door --dir /path/to/mygame --dropfile /path/to/DOOR32.SYS --socket-port {srvPort}
```

Windows Synchronet also passes the inherited socket descriptor:

```
nc-door --dir /path/to/mygame --dropfile %f --socket-descriptor %H
```

Linux Synchronet is validated with the same `DOOR32` dropfile shape. If your Synchronet install mangles a long native `cmd=` line, keep the full `nc-door --dir ... --dropfile ...` invocation inside a tiny wrapper and pass only `%f` into that wrapper.

WWIV is documented separately as a validated Linux `CHAIN.TXT` path:

```
nc-door --dir /path/to/mygame --dropfile /path/to/CHAIN.TXT
```

Treat native Windows WWIV as unvalidated for now. In one local Windows 11 test with an older 2023 WWIV Windows package, Defender quarantined the downloaded binary and `bbs.exe` then crashed before login in both remote and local-node flows.

For the exact launcher setups, see `docs/sysop/bbs/mystic-bbs-setup.md`, `docs/sysop/bbs/enigma-bbs-setup.md`, `docs/sysop/bbs/synchronet-bbs-setup.md`, and `docs/sysop/bbs/wwiv-bbs-setup.md`.

8. File-Based Turn Submission

Use this when a player needs to submit a KDL turn file directly:

1. Validate the file without writing:

```
nc-game submit-turn --check --dir /path/to/mygame --player 1 --file /path/to/player1-turn.kdl
```

2. Apply it:

```
nc-game submit-turn --dir /path/to/mygame --player 1 --file /path/to/player1-turn.kdl
```

The `--player` value must match the `turn player=...` header in the file. If any order in the file is invalid, the entire submission is rejected and nothing is written. This is a direct apply command, not a queue or upload inbox.

A minimal turn file:

```
turn player=1 year=3000
tax rate=37
```

A fuller example:

```
turn player=1 year=3000

tax rate=37

planet record=16 {
  build points=4 kind="scout"
}

fleet record=1 {
  order speed=3 kind="scout_system" x=16 y=13
}

message to=2 subject="Border" body="Watching the north lane."
```

A `seek_home` or `salvage` order may leave out `x` and `y`. The fleet then heads for the owned world nearest it, the world the game would pick.

`speed value=N` inside a `fleet` block changes only that fleet's speed, as the in-game speed change does.

For the full node reference and schema, see `docs/sysop/rust/turn-kdl.md`.

9. Turn Processing and Maintenance

Run yearly maintenance with:

```
nc-sysop maint /path/to/mygame [turns]
```

`nc-sysop maint` advances the campaign in `ncgame.db` immediately. It does not change the schedule fields. NC does not schedule maintenance by itself.

Use one of these methods:

- a `systemd` timer
- `cron`
- a BBS event runner
- or manual sysop operation

To put one game on a real schedule, turn scheduling on and set the first due time yourself:

```
nc-sysop settings set --dir /srv/nc/games/friday-night --maintenance-enabled on --
maintenance-interval-minutes 10080 --maintenance-next-due 1775347200
```

That example enables weekly scheduling. `10080` is seven days in minutes. `1775347200` is only a sample Unix timestamp. Replace it with the first due time you actually want.

Treat these schedule fields as optional metadata. They do not create their own timer. They are not BBS `config.kdl` fields. If you run a direct localhost game, you can ignore them and schedule `nc-sysop maint /path/to/mygame` yourself.

Hosted games on the `nc-host` path, which is not part of the live workflow yet, keep their own schedule in `hosted.db`. `nc-host serve` runs a game's maintenance when that schedule is due, and `nc-host maint /path/to/game [turns]` runs it by hand. Both apply the turns players submitted over the relay, run the same yearly pass as `nc-sysop maint`, and send each claimed seat its new state. A game that has never been scheduled is armed one interval from the moment the daemon first sees it.

10. Player Management

Non-BBS campaigns keep player reservations and the carried-forward inactivity-related state in `ncgame.db`. BBS door campaigns do not use a separate inactivity block in `config.kdl`; caller idle handling belongs to the BBS software.

For direct DB-only campaigns, the default inactivity autopilot threshold is `3` turns. Set `inactivity_autopilot_after_turns=0` if you want it disabled. A player counts as active for the year by either reaching the live `nc-game` menus or successfully applying a `submit-turn` file.

A defeated player retains one final review-only login. If the empire has no planets, the next maintenance turns it into a computer-controlled rogue empire. The former player's identity and final-review state remain in the store; this does not reopen the seat or restore orders. A planetless rogue patrols with its warships and does not contend for the throne.

10.1. Reserving Seats

To reserve empire slots for specific BBS users, edit the per-game BBS `config.kdl` or use:

```
nc-sysop settings reserve --dir /path/to/mygame --player 1 --alias SYSOP
nc-sysop settings reserve --dir /path/to/mygame --player 2 --alias NightShade
```

Each `seat` entry binds one 1-based empire slot to one caller alias. Alias matching is ASCII case-insensitive, so `NightShade`, `nightshade`, and `NIGHTSHADE` are treated as the same reservation.

With reservations in place, launch `nc-door` with `--dropfile` and let the caller alias choose the empire automatically:

```
nc-door --dir /path/to/mygame --dropfile /path/to/DOOR32.SYS
```

IMPORTANT:

- if the caller alias is reserved, `--dropfile` alone is enough
- if the caller alias is not reserved and already matches a stored joined player handle, that caller resumes the matching empire automatically
- if the caller alias is not reserved and does not match an existing joined player handle, the caller starts on the BBS first-time menu
- if a reservation conflicts with an already-stored different player handle, `nc-door` will stop with a clear error so the sysop can reconcile the reservation and the runtime state
- if no open unreserved empires remain, `J` from the BBS first-time menu is refused and the caller stays on that menu

Reserving a seat does not join or pre-name the empire by itself. It only routes the caller to the intended slot. The usual first-time join flow still claims an empire only on successful join confirmation, and that first successful join records the caller alias for later dropfile logins.

For localhost and direct console play, use `nc-game` and follow the localhost setup section earlier in this manual instead of the BBS dropfile path.

11. Terminology

- **game directory:** The directory containing `ncgame.db` for one running game. Pass it to all tools with `--dir`.
- **`nc-sysop`:** The Rust command-line sysop tool for campaign creation, maintenance, and settings management.
- **`nc-game`:** The Rust TUI player client for direct localhost sessions.
- **`nc-door`:** The Rust BBS door entrypoint. It runs the same game flow as `nc-game`, but it is the staged binary for Windows and Linux BBS hosts.

- **`ncgame.db`:** The SQLite database that is the runtime source of truth for the Rust engine.
- **non-BBS campaign settings:** The sysop-managed runtime policy rows stored in `ncgame.db` for direct `nc-game` campaigns. They control game name, the default compiled-in color set, maintenance metadata, optional alias reservations, and a small set of carried-forward classic setup fields.
- **`config.kdl`:** Present only for BBS door campaigns. It holds `players` and optional seat `reservations`.

12. CLI Reference

12.1. nc-sysop

```
nc-sysop <subcommand> [options]
```

- **`new-game`:** Create a new campaign directory. Direct `nc-game` campaigns use `--name`, `--players`, and `--seed`. BBS campaigns use `new-game --bbs` with a minimal per-game `config.kdl`.
- **`settings show|set|reserve|unreserve`:** Inspect or edit non-BBS runtime policy in `ncgame.db`, or edit BBS seat reservations in per-game `config.kdl`.
- **`maint`:** Run one or more maintenance turns against `ncgame.db`.

12.2. nc-game and nc-door

```
nc-game --dir <game_dir> [--player <1-based index>] [options]
nc-door --dir <game_dir> --dropfile <path> [options]
nc-game submit-turn [--check] --dir <game_dir> --player <record> --file <turn.kdl>
```

Use `nc-game` for direct localhost sessions. Use `nc-door` for BBS door launches. The interactive flags below apply to both unless a host guide says otherwise.

Interactive client flags:

- **`--dir <path>`:** Game directory containing `ncgame.db`. Required.
- **`--player <N>`:** 1-based empire index for direct localhost play. Normal BBS dropfile launches can omit it.
- **`--encoding <utf8|cp437>`:** Output encoding. Default: `utf8`. Use `cp437` for BBS or door mode.
- **`--color-mode <ansi16|256|truecolor|auto>`:** Color depth. Default: `auto` from the environment. CP437 mode defaults to `ansi16`.
- **`--dropfile <path>`:** Parse a BBS drop file (`DOOR32.SYS`, `DOOR.SYS`, or `CHAIN.TXT`). It supplies the alias and timeout, defaults encoding to `cp437`, and resolves the player seat through BBS `config.kdl` reservations or stored joined-player aliases.
- **`--socket-port <value>`:** Connect back to a localhost door socket. Mainly for ENiGMA½ `abracadabra` in `socket` mode.
- **`--socket-descriptor <value>`:** Native Windows door socket handle. Mainly for Synchronet-style socket door launches.
- **`--timeout <minutes>`:** Session time limit in minutes. It overrides any drop file value. Door mode ends the session when it runs out.
- **`--export-root <path>`:** Optional map or export staging root for local or BBS file handoff.
- **`--queue-dir <path>`:** Override turn queue directory. Default: `<game_dir>/queue`.

`submit-turn` flags:

- **`--check`:** Validate the KDL file without mutating the campaign.
- **`--dir <path>`:** Game directory containing `ncgame.db`. Required.
- **`--player <N>`:** 1-based empire index. Required, and it must match the KDL header.
- **`--file <path>`:** Turn submission KDL file to validate or apply. Required.

> **NOTE:** `nc-game submit-turn` is all-or-nothing. If any command in the file is invalid, the entire submission is rejected and nothing is written to `ncgame.db`.